

Universidad del Valle de Guatemala

Facultad de Ingeniería

Laboratorio 7

Entrega final

CC3104 – Reinforcement Learning

Integrantes

Andy Fuentes — 22944

Diederich Solís — 22952

Diego Linares — 221256

Repositorio

https://github.com/DiederichSolis/lab7_RL

Fecha: 28 de septiembre de 2026

1. Task 1.1: análisis de DQN estándar

1.1(a) Estado continuo de 256 dimensiones

Sí es razonable usar DQN desde el punto de vista arquitectónico: una red puede aproximar $Q(s, a; \mathbf{w})$ aunque $s \in \mathbb{R}^{256}$ no pueda enumerarse en una tabla. Como las 256 entradas son métricas numéricas y no píxeles, usaríamos una red totalmente conectada (MLP), por ejemplo, dos capas ocultas con ReLU y una capa final de cuatro salidas. Cada salida estima el valor de una acción. Es importante normalizar las variables de entrada, porque los conteos de tráfico y otras métricas pueden tener escalas muy distintas.

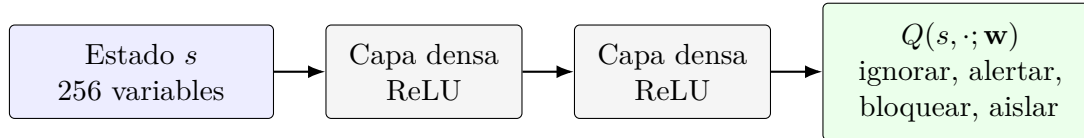


Figura 1: Esquema propuesto para aproximar los cuatro valores Q a partir del vector de estado.

La Figura 1 muestra por qué no hace falta una red convolucional: no se ha definido una estructura espacial como la de una imagen. La conveniencia del método todavía depende de que las características resuman información temporal suficiente y de que exista una forma segura de obtener transiciones y recompensas para aprender.

1.1(b) Cuatro acciones discretas

Las acciones $\mathcal{A} = \{\text{ignorar, alertar, bloquear, aislar}\}$ son finitas y discretas. Por eso DQN y sus variantes pueden producir directamente un valor por acción y elegir

$$a^*(s) = \arg \max_{a \in \mathcal{A}} Q(s, a; \mathbf{w}). \quad (1)$$

No se necesita un algoritmo para acciones continuas. Esta compatibilidad del espacio de acciones no demuestra que DQN sea seguro o eficaz en producción: aislar un segmento tiene un costo distinto al de emitir una alerta, y ambos deben reflejarse en la recompensa y la evaluación.

1.1(c) Eventos críticos raros y Experience Replay

Sea $p < 0.001$ la proporción de pasos con intrusión real. Bajo el supuesto simplificador de eventos independientes y una distribución estable, si el buffer contiene N transiciones, el número K de transiciones críticas satisface $K \sim \text{Binomial}(N, p)$ y $\mathbb{E}[K] = Np$. En tráfico real puede haber ráfagas y cambios de distribución; por ello, esta expresión es una referencia, no una garantía sobre el contenido del buffer. Al ser de capacidad fija, el buffer también expulsa transiciones antiguas: puede llegar a contener $K = 0$ eventos críticos.

Condicionado a un buffer con K eventos críticos, un minibatch uniforme de tamaño B sin reemplazo contiene al menos uno con probabilidad

$$\Pr(\text{al menos un crítico} \mid K) = 1 - \frac{\binom{N-K}{B}}{\binom{N}{B}}, \quad \mathbb{E}[\text{críticos por lote} \mid K] = B \frac{K}{N}. \quad (2)$$

Para ilustrar, si un buffer representativo tiene $K/N < 0.001$ y $B = 64$, el número esperado es menor que 0.064 por lote. Además, $\Pr(\text{al menos un crítico}) \leq \mathbb{E}[\text{críticos por lote}]$, así que más del 93.6 % de esos lotes no incluiría una intrusión. Las consecuencias de ignorarla tendrían poca presencia en el aprendizaje.

PER aborda esa dilución al dar mayor probabilidad a las transiciones con error de diferencia temporal (TD) alto [2]:

$$\delta_i = y_i - Q(s_i, a_i; \mathbf{w}), \quad p_i = |\delta_i| + \varepsilon, \quad P(i) = \frac{p_i^\alpha}{\sum_{j \in \mathcal{D}} p_j^\alpha}, \quad \varepsilon > 0, \alpha \in [0, 1]. \quad (3)$$

Aquí y_i es el target temporal de la transición i . Si una intrusión poco frecuente produce un error TD grande, PER permite reutilizarla más que el muestreo uniforme. $\alpha = 0$ recupera el muestreo uniforme; pesos de importancia proporcionales a $(NP(i))^{-\beta}$ pueden compensar el sesgo del nuevo muestreo. PER no crea eventos ausentes ni garantiza priorizar un caso crítico cuyo error TD sea bajo: habría que vigilar además la cobertura de incidentes en el buffer.

1.1(d) Recompensa y falsos positivos

Si se premiara únicamente reducir falsos positivos, la política más sencilla sería *ignorar siempre*: no bloquearía tráfico legítimo, pero dejaría pasar intrusiones. El 8 % de falsos positivos del sistema de reglas sirve como contexto, no determina por sí solo los costos de una función de recompensa.

Una propuesta es definir $z_t = 1$ cuando se confirma una intrusión, $h_t \in [0, 1]$ como su gravedad, $e_t \in [0, 1]$ como fracción contenida y $m_t \in [0, 1]$ como fracción del daño que se materializó. Para la acción a_t :

$$r_t = \underbrace{20z_th_te_t}_{\text{contención útil}} - \underbrace{1000z_th_tm_t}_{\text{daño por intrusión}} - \underbrace{(1 - z_t)C_{\text{FP}}(a_t)}_{\text{intervención sobre tráfico legítimo}} - \underbrace{C_{\text{op}}(a_t)}_{\text{costo operativo}}. \quad (4)$$

Acción a	$C_{\text{FP}}(a)$	$C_{\text{op}}(a)$
Ignorar	0	0
Alertar	2	0.2
Bloquear temporalmente	6	1
Aislar segmento	20	3

Estos números son **ejemplos para discutir prioridades, no parámetros validados**. El daño de una intrusión grave recibe el mayor peso; una intervención errónea cuesta más cuanto más interrumpe la operación; y el costo operativo evita acciones innecesarias incluso si hay ataque. La contención efectiva aporta un beneficio menor que la pérdida asociada al daño grave. En una implementación habría que definir cómo observar e_t y m_t , que pueden conocerse con retraso, y ajustar la escala con especialistas y datos de incidentes. La recompensa debe favorecer decisiones según el riesgo del estado, no una acción fija para todos los pasos.

2. Task 1.2: análisis de Double DQN

1.2(a) Sobreestimación y consecuencia operacional

Supongamos que las estimaciones de una red son $\hat{Q}(s', a) = Q(s', a) + \epsilon_a$, con error medio cero para cada acción. Como el máximo es una función convexa,

$$\mathbb{E}[\max_a \hat{Q}(s', a)] \geq \max_a \mathbb{E}[\hat{Q}(s', a)] = \max_a Q(s', a). \quad (5)$$

El máximo tiende a escoger errores positivos junto con acciones aparentemente buenas. En DQN la misma red objetivo selecciona y evalúa la acción del siguiente estado, de modo que ese valor optimista entra al target y puede propagarse mediante *bootstrapping* [2]. Double DQN reduce esta fuente de sesgo al separar selección y evaluación; no elimina todo error de estimación.

Si en estados relevantes se sobrevalora *bloquear*, el agente podría bloquear tráfico legítimo y causar interrupciones y falsos positivos. Si se sobrevalora *ignorar*, podría omitir ataques, retrasar la respuesta

y permitir mayor daño. La sobreestimación del máximo no implica que todas las acciones estén sobreestimadas en cada estado: estas son consecuencias posibles cuando el error favorece una acción concreta.

1.2(b) Targets y cambio exacto

Para una transición (s, a, r, s', d) , donde $d = 1$ si el episodio terminó, sea $\mathbf{w}$ la red en línea y $\mathbf{w}^-$ la red objetivo. Los targets son:

$$y_{\text{DQN}} = r + \gamma(1 - d) \max_{a' \in \mathcal{A}} Q(s', a'; \mathbf{w}^-), \quad (6)$$

$$a_{\text{online}}^* = \arg \max_{a' \in \mathcal{A}} Q(s', a'; \mathbf{w}),$$

$$y_{\text{Double}} = r + \gamma(1 - d) Q(s', a_{\text{online}}^*; \mathbf{w}^-). \quad (7)$$

En (6), $\mathbf{w}^-$ tanto selecciona mediante $\max$ como evalúa. En (7), $\mathbf{w}$ *solo selecciona* a_{online}^* y $\mathbf{w}^-$ *evalúa* esa acción. En ambos casos se ajusta $\mathbf{w}$ para aproximar $Q(s, a; \mathbf{w})$ al target, por ejemplo minimizando $(y - Q(s, a; \mathbf{w}))^2$, y se sincroniza periódicamente la red objetivo. El único cambio algorítmico de implementación es el cálculo del target: reemplazar el máximo evaluado por la red objetivo por una selección con la red en línea y una evaluación con la red objetivo. No se agrega una tercera red [2].

1.2(c) Estados ambiguos y claramente maliciosos

En tráfico *borderline*, los valores reales de varias acciones pueden estar próximos. Entonces un error pequeño puede cambiar cuál parece mejor, y el máximo de DQN puede amplificar la estimación optimista de una acción costosa. El riesgo es especialmente delicado cuando la decisión alterna entre ignorar y bloquear: ambos errores tienen consecuencias operativas distintas.

En estados claramente maliciosos, si una acción de contención supera ampliamente a las demás en valor real, errores pequeños tienen menos probabilidad de cambiar la acción elegida. Puede seguir habiendo sobreestimación del valor numérico, pero su efecto sobre la decisión suele ser menor. Esta comparación depende de la separación entre valores, la incertidumbre de las observaciones y los costos; no demuestra que todos los estados maliciosos sean fáciles de clasificar.

3. Task 2.1: implementación en LunarLander

2.1(a) Entorno y condiciones comunes

Se verificó que Gymnasium 0.29.1 acepta **LunarLander-v2**, con observación continua de ocho componentes y cuatro acciones discretas; no fue necesario cambiar el identificador. El cuaderno adjunto implementa DQN y Double DQN desde cero con PyTorch. Cada red tiene dos capas ocultas de 128 neuronas ReLU y cuatro salidas. Los tres entrenamientos usan un buffer de 50 000, lotes de 64, sincronización de la red objetivo cada 500 pasos del entorno y $\epsilon_t = 1 - 0.95 \min(t/10\,000, 1)$. Se ejecutaron 500 episodios con una semilla (2026) por método, manteniendo las mismas semillas de reinicio por episodio.

Como elecciones adicionales se usaron $\gamma = 0.99$, Adam con tasa 5×10^{-4} , 1 000 transiciones antes de actualizar, un gradiente cada cuatro pasos y recorte de norma del gradiente a 10. Las condiciones son iguales para los tres métodos, aunque sus trayectorias y números de pasos difieren al aprender políticas distintas. En una truncación por tiempo se corta el episodio, pero el target conserva el término de valor futuro. La actualización de DQN y Double DQN usa la pérdida $\frac{1}{B} \sum_i (y_i - Q(s_i, a_i; \mathbf{w}))^2$, con los targets de (6) y (7).

2.1(b) Registros por episodio y PER

Antes de entrenar se fijaron 100 estados tomados de trayectorias aleatorias independientes. Al final de cada episodio se registra su recompensa total, $\bar{Q} = \frac{1}{100} \sum_{s \in S_{\text{fijo}}} \max_a Q(s, a; \mathbf{w})$ y el promedio de los targets de los minibatches realmente usados. En los primeros episodios, sin actualizaciones, el último registro queda vacío. El mismo conjunto S_{fijo} se usa en las tres variantes.

Sobre Double DQN se añadió PER proporcional. Para cada transición, $p_i = |\delta_i| + 10^{-5}$ y $P(i) = p_i^{0.6} / \sum_j p_j^{0.6}$; las prioridades se actualizan con el error TD anterior al paso de gradiente. En la pérdida se emplean pesos de importancia

$$w_i = \frac{(NP(i))^{-\beta}}{\max_j (NP(j))^{-\beta}}, \quad \beta : 0.4 \longrightarrow 1.0 \quad \text{linealmente entre los episodios 1 y 500.} \quad (8)$$

Las transiciones nuevas reciben inicialmente la mayor prioridad observada. El cuaderno incluye verificaciones de la diferencia entre targets y de las probabilidades y pesos de PER; sus celdas se ejecutaron de principio a fin. Las curvas y cifras siguientes proceden de esa ejecución, no de datos simulados para las figuras.

4. Task 2.2: resultados y análisis

Método	Pasos	Primer $\text{MM20} \geq 200$	Recomp. final (20)	$\bar{Q}$ final (20)
DQN	260 790	269	243.16	86.77
Double DQN	317 678	397	237.75	54.24
Double DQN + PER	252 426	306	234.61	27.13

Cuadro 1: Medidas observadas en una semilla y 500 episodios por variante. MM20 es la media móvil de 20 episodios; el primer cruce de $+200$ es un indicador descriptivo, no una prueba de convergencia estable.

2.2(a) Recompensa: DQN frente a Double DQN

La Figura 2 muestra las recompensas de cada episodio y sus medias móviles de 20. Usando como referencia el primer cruce de $\text{MM20} \geq 200$, DQN avanzó antes (episodio 269) que Double DQN (397).

También obtuvo una media ligeramente mayor en los 20 episodios finales: 243.16 frente a 237.75. Las curvas oscilan y un cruce aislado no significa convergencia definitiva. Esta corrida no respalda una ventaja de aprendizaje de Double DQN, aunque la teoría solo propone reducir el sesgo del máximo; no garantiza mejor recompensa en toda semilla o etapa.

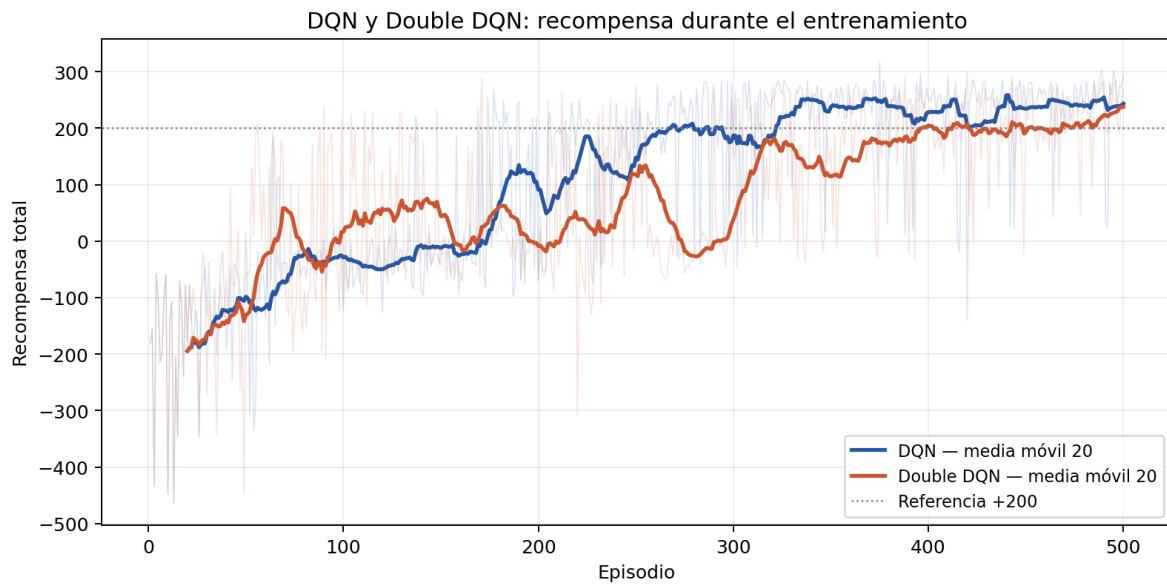


Figura 2: Recompensa real por episodio de DQN y Double DQN; líneas gruesas: media móvil de 20.

2.2(b) Valor Q y targets

En los mismos 100 estados, $\bar{Q}$ de DQN fue mayor que el de Double DQN en 428 de 500 episodios y en todos los episodios 101–500. En los últimos 20, las medias fueron 86.77 y 54.24, respectivamente; los targets promedio fueron 53.74 y 24.55. La Figura 3 presenta ambos registros. El patrón es compatible con la sobreestimación que motiva Double DQN, pero $\bar{Q}$ no es un error estimado frente a retornos verdaderos de esos estados. Las políticas también visitaron distribuciones distintas, por lo que no atribuimos toda la separación a sesgo algorítmico.

2.2(c) Double DQN uniforme frente a PER

La Figura 4 compara las dos formas de replay. PER cruzó $MM20 \geq 200$ en el episodio 306, antes que el uniforme (397). La mayor ventaja apareció entre los episodios 251–400: recompensa media 198.44 con PER y 117.44 sin PER, una diferencia de 81.00 puntos. Sin embargo, PER fue peor en 1–100 (−131.73 frente a −72.40), 101–250 (7.21 frente a 41.00) y 401–500 (193.82 frente a 206.96). Sus últimos 20 episodios promediaron 234.61 frente a 237.75. Por tanto, esta ejecución muestra una aceleración intermedia, no una mejora sostenida ni una mayor recompensa final.

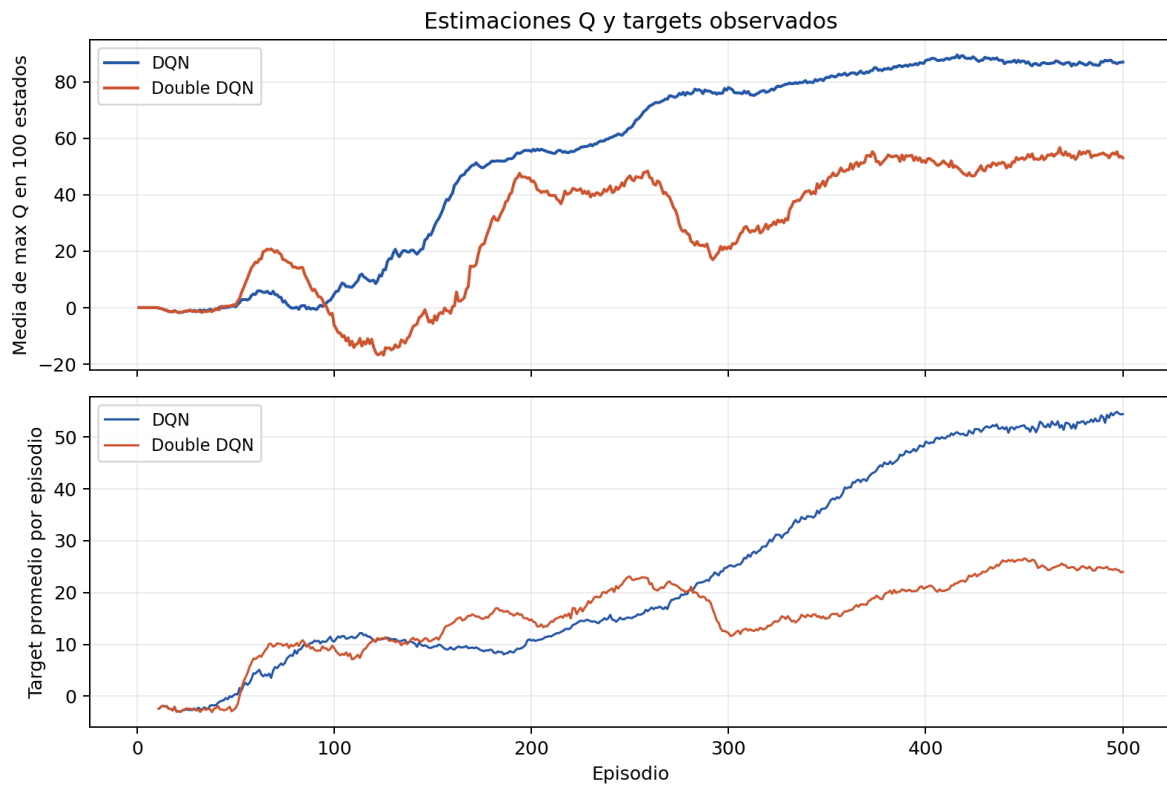


Figura 3: Media de $\max_a Q$ en los 100 estados fijos y target promedio de las actualizaciones por episodio.

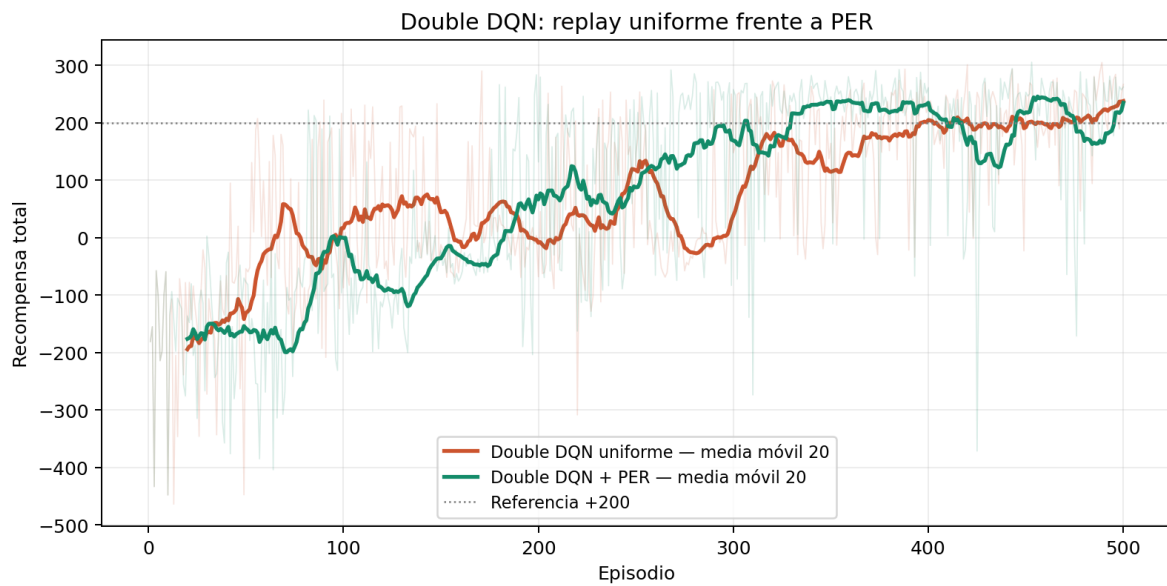


Figura 4: Comparación observada de Double DQN con muestreo uniforme y con PER.

2.2(d) Relación con intrusiones raras

En Task 1.1(c) vimos que, con menos de 0.1 % de intrusiones, un lote uniforme de 64 casi nunca contendría un evento crítico si el buffer representa esa frecuencia. PER podría reutilizar más los casos con error TD alto, pero solo después de que ingresen al buffer; además, un caso grave con error TD bajo podría no priorizarse. La ventaja intermedia de 81.00 puntos observada en LunarLander muestra que cambiar el muestreo puede alterar la velocidad de aprendizaje, mientras que sus fases desfavorables impiden afirmar que siempre ayuda. LunarLander no contiene intrusiones, falsos positivos ni costos de aislamiento y **no demuestra** rendimiento en una red real. Se requieren pruebas con rareza y costos representativos, por clase de ataque y con varias semillas.

5. Task 2.3: investigación y dictamen técnico

2.3(a) Artículo seleccionado y resumen técnico

Mahjoub, Hamdi, Alkanhel, Mohamed y Ejbalí (2024), “An adversarial environment reinforcement learning-driven intrusion detection algorithm for Internet of Things”, *EURASIP Journal on Wireless Communications and Networking*, artículo 21, DOI: [10.1186/s13638-024-02348-6](https://doi.org/10.1186/s13638-024-02348-6) [3].

El trabajo aborda la clasificación de tráfico IoT normal y malicioso con el conjunto BoT-IoT. Usa un clasificador entrenado por refuerzo y un entorno adversarial que elige muestras difíciles del conjunto etiquetado para atender el desbalance. Los autores describen su implementación final como basada en Double DQN, junto con pérdida Huber y estrategias de exploración separadas. El uso de Double DQN es coherente con la intención de reducir el sesgo de estimación, pero el artículo no mide ese efecto por separado. Su acción es una *etiqueta de clase*, no ignorar, alertar, bloquear o aislar. En la comparación multiclase reportan exactitud 0.9427 y F1 0.9433 frente a 0.8164 y 0.8050 de un MLP; la probabilidad de falsa alarma es 0.055 frente a 0.175. Pero la clase *Theft* obtiene F1 0.0491, señal de que el promedio oculta un problema importante en una clase concreta.

El muestreo adversarial intenta atender clases difíciles, aunque el artículo deja PER como trabajo futuro y no presenta una ablación que aísle el aporte de Double DQN. Además, en su extracción de BoT-IoT el tráfico normal es minoritario, una distribución distinta al escenario empresarial con intrusiones inferiores a 0.1 %. Su recompensa de clasificación correcta no representa daños de una omisión ni interrupciones por bloqueo. Por ello, el artículo aporta evidencia de clasificación en un banco de datos, pero no resuelve por sí solo la rareza, los costos de acción y la seguridad operacional identificados en Task 1.

2.3(b) Dictamen a la dirección

Recomendamos continuar con un *prototipo de Double DQN con PER* como candidato para el sistema de cuatro acciones, sin autorizar todavía decisiones automáticas de bloqueo o aislamiento. En LunarLander, PER adelantó a Double DQN uniforme en el primer cruce de la media móvil de +200 (episodios 306 frente a 397) y mejoró la fase 251–400, pero su recompensa final fue ligeramente menor (234.61 frente a 237.75); DQN estándar también rindió mejor en esta única semilla. Por tanto, la elección se justifica como hipótesis a evaluar frente a intrusiones raras, no como superioridad comprobada. Antes de producción hace falta construir un entorno o simulador con eventos críticos y costos reales, calibrar la recompensa con especialistas, medir por clase falsos positivos, falsos negativos y daño, repetir con varias semillas y tráfico cambiante, comparar contra las reglas actuales y empezar con evaluación supervisada sin ejecución automática de acciones costosas. El artículo de Mahjoub *et al.* apoya estudiar métodos de refuerzo para clasificación, pero no valida el control de respuesta de esta empresa [3].

Referencias

- [1] CC3104 – Reinforcement Learning. *Laboratorio 7*, guía de la actividad, 2026, Tasks 1 y 2.
- [2] CC3104 – Reinforcement Learning. *Deep RL Parte 1: DQN y Variantes*, diapositivas de clase, 2026.
- [3] C. Mahjoub, M. Hamdi, R. I. Alkanhel, S. Mohamed y R. Ejbali. “An adversarial environment reinforcement learning-driven intrusion detection algorithm for Internet of Things.” *EURASIP Journal on Wireless Communications and Networking*, 2024, art. 21. [doi:10.1186/s13638-024-02348-6](https://doi.org/10.1186/s13638-024-02348-6).